

CURSOR HUNTER

프로토타입 병렬 개발 가이드

특성 / 전투 화면 담당자가 서로 기다리지 않고 구현한 뒤, 하나의 런타임 루프로 통합하는 방법

기준 문서: v0.0.4 기획, 연결 계약 v0, 모듈 소유권 문서

프로토타입 목표: 맵 -> 스폰 -> 커서 공격 -> 히트 -> 사망 -> 가넷 획득 -> 시간 종료 -> 결과창

한 줄 원칙: 특성 담당자는 능력치를 계산하고, 전투 담당자는 전달받은 스냅샷으로 공격을 실행한다.

1. 이 문서에서 구현할 범위

이번 프로토타입은 60초 일반 Field 한 판을 완주하는 것에 집중한다.

포함	프로토타입 기준
맵	단순 배경과 몬스터 생성 영역
몬스터	원형 슬라임 1종, 고정 데이터 주입
공격	마우스 위치와 몬스터 겹침, 좌클릭 1회 피해
처치	HP 감소, 사망, 처치 수와 런 가넷 누적
종료	60초에 생성과 공격 중지, 생존 몬스터 보상 없이 제거
결과	처치 수, 획득 가넷, 전투 시간 표시

이번 단계에는 특성 구매, 보스, 스킬, 자동 공격, 영구 저장, 최종 아트를 넣지 않는다. 단, 다음 단계에서 특성 수치를 연결할 수 있는 입력 지점은 처음부터 만든다.

2. 모듈별 소유권

담당	소유하는 결과	독립 개발 방식
특성 / Progression	특성 정의, 구매 상태, 프로필, 유효 능력치 계산	가짜 프로필과 가짜 재화로 특성 Sandbox 실행
전투 / Combat	커서 입력, 판정, 피해, 스폰, 타이머, 처치, 전투 HUD	고정 CombatSnapshot으로 Combat Sandbox 실행
연결 / App	런 시작, 화면 전환, 스냅샷 전달, 결과 표시	두 모듈을 마지막에 연결
공통 / Contracts	ID, 값 객체, 런 입출력 타입	양쪽 구현보다 먼저 합의하고 병합

참조 방향은 다음을 유지한다.

```
Contracts <- Data
Contracts <- Progression
Contracts <- Combat
Contracts + Data + Progression + Combat <- App
```

Combat은 Progression이나 Data 구현을 직접 참조하지 않는다. Progression도 Combat 구현 클래스나 씬을 참조하지 않는다.

3. 공유 계약: 유일한 연결 지점

두 담당자가 공유하는 것은 특성 화면이나 전투 오브젝트가 아니라 순수 데이터 스냅샷이다.

```

RunRequest
- runId
- seed
- durationSeconds = 60

CombatSnapshot
- attack: long
- radiusPixels: int
- hitsPerBundle: int
- autoEnabled: bool
- autoBundlesPerSecond: int

SpawnSnapshot
- monsterId
- hp: long
- spawnInterval
- reward table

RunResult
- runId
- end reason
- elapsed seconds
- kill counts
- earned reward

```

계약에는 ScriptableObject, GameObject, 씬 참조, 저장 파일 경로를 넣지 않는다. 컬렉션은 읽기 전용 복사본으로 전달한다.

프로토타입에서는 autoEnabled=false, autoBundlesPerSecond=0으로 두고 공격력 / 범위 / 타격 수만 사용한다.

4. 능력치의 데이터 흐름

```

TraitDefinition
-> Progression Profile
-> TraitCalculator
-> CombatSnapshot
-> App
-> CombatRunController
-> CursorAttackController

```

특성 담당자는 구매한 특성 ID와 레벨을 보유하고, 그 상태에서 최종 공격력과 범위를 계산한다.

전투 담당자는 TraitId나 특성 트리를 해석하지 않는다. attack=10, radiusPixels=18처럼 계산이 끝난 값만 사용한다.

```

특성 구매 전
attack = 10, radius = 18

특성 구매 후 다음 런
attack = 15, radius = 24

```

특성 구매 중인 현재 런에는 새 값을 적용하지 않는다. 다음 런을 시작할 때 새 스냅샷을 생성한다.

5. 특성 담당자의 작업

구현 대상

- TraitDefinition: 특성 ID, 효과 종류, 효과값, 비용, 선행 조건
- Profile: 구매한 특성 ID 또는 레벨과 가넷 보유량

- TraitCalculator: Profile을 유효 CombatSnapshot으로 변환
- 특성 Sandbox: 가짜 프로필로 현재값과 다음값 표시
- Progression 테스트: 구매 전후 공격력 / 범위 변화, 재화 부족, 중복 구매 방지

프로토타입 테스트 값

```
Profile
- garnet = 100
- attack = 10
- radiusPixels = 18
- hitsPerBundle = 1

구매 예시
- PWR_01 -> attack 증가
- RANGE_1 -> radiusPixels 증가
```

특성 화면은 CursorAttackController를 직접 호출하지 않는다. 구매가 성공하면 Profile을 갱신하고, App이 다음 런에서 Snapshot을 요청하도록 한다.

6. 전투 담당자의 작업

구현 대상

- CombatRunController: 런 시작, 타이머, 종료 상태
- MonsterSpawner: 고정 seed, 생성 주기, 생성 영역
- MonsterRuntime: HP, 피격, 사망, 풀 반환
- CursorAttackController: Screen 좌표 -> World 좌표, 겹침 판정, 피해 적용
- RunAccumulator: 처치 수, 런 가넷, 유효 피해
- CombatHudPresenter: 타이머, 가넷, 처치 수
- ResultPanelController: 런 결과 표시

전투 불변 규칙

- 커서와 살아 있는 몬스터가 겹친 대상 모두에게 피해를 준다.
- 이동이나 마우스 홀드만으로는 공격하지 않는다.
- 몬스터 HP와 보상은 프리팹에 복제하지 않고 SpawnSnapshot으로 주입한다.
- t=60에는 생성과 공격을 처리하지 않는다.
- 시간 종료 시 살아 있는 몬스터는 보상 없이 제거한다.
- 같은 몬스터의 사망과 보상 처리는 한 번만 수행한다.
- 전투는 영구 지갑이나 저장 파일을 직접 변경하지 않는다.

7. Sandbox와 씬 경계

```
Assets/CursorHunter/Combat/Scenes/CombatSandbox.unity
Assets/CursorHunter/Progression/Scenes/ProgressionSandbox.unity
Assets/CursorHunter/App/Scenes/Bootstrap.unity
```

Combat Sandbox에는 카메라, EventSystem, 맵, 슬라임, HUD를 둔다. Progression Sandbox에는 가짜 Profile, 특성 노드, 구매 UI를 둔다.

두 Sandbox를 Additive로 함께 로드하지 않는다. 통합할 때는 각 모듈의 루트 프리팹을 App의 단일 런타임 씬에 연결한다.

각 담당자는 자신의 폴더와 Sandbox만 수정한다. 같은 씬, 같은 프리팹, 공급사 원본 에셋을 동시에 편집하지 않는다.

8. 병렬 작업 순서

1단계: 계약 먼저

공동으로 ID, 단위, 기본값, 런 수명 규칙을 설정하고 Contracts/Runtime에 반영한다.

```
attack = long
radiusPixels = int
durationSeconds = 60
GARNET = 프로토타입 런 보상 ID
```

2단계: 각자 Sandbox 개발

전투 담당자는 FixedCombatSnapshot으로 전투를 완성한다. 특성 담당자는 가짜 Profile로 구매와 계산을 완성한다.

두 작업은 상대 모듈의 코드가 없어도 실행되어야 한다.

3단계: 계약 기반 테스트

전투 테스트는 공격력 10과 20, 범위 18과 30의 서로 다른 Snapshot을 넣었을 때 결과가 달라지는지 확인한다.

특성 테스트는 구매 전후 Snapshot 값이 바뀌고, 구매하지 않은 현재 런의 Snapshot은 바뀌지 않는지 확인한다.

4단계: App 통합

App이 다음 순서로 연결한다.

```
Profile 읽기
-> CombatSnapshot 생성
-> RunRequest 생성
-> Combat.Start(request, snapshot)
-> RunResult 수신
-> 결과창 표시
```

통합 후에도 Combat이 Progression을 직접 참조하면 안 된다.

9. 통합 완료 기준

- 새 실행 후 60초 일반 런을 시작할 수 있다.
- 슬라임 생성, 좌클릭 공격, HP 감소, 사망, 처치 수 누적이 동작한다.
- 사망한 슬라임마다 가넷이 한 번만 누적된다.
- 60초에 생성과 공격이 멈추고 결과창이 열린다.
- 결과창에는 처치 수와 런 가넷이 표시된다.
- 종료 후 클릭이 결과창 위의 전투로 전달되지 않는다.
- 전투는 고정 Snapshot만으로 실행된다.
- 공격력 또는 범위가 다른 Snapshot을 넣으면 전투 결과가 달라진다.
- 특성 담당자의 구현 없이도 Combat Sandbox가 실행된다.
- Combat 담당자의 구현 없이도 특성 Sandbox가 실행된다.

10. 하지 말아야 할 것

- Combat에서 Progression Service를 싱글턴으로 호출하지 않는다.
- 특성 화면에서 Combat 오브젝트의 필드를 직접 수정하지 않는다.
- 커서 프리팹에 영구 공격력과 범위를 저장하지 않는다.
- 몬스터 프리팹에 최종 HP와 보상을 하드코딩하지 않는다.
- Contracts에 UnityEngine 타입이나 씬 참조를 넣지 않는다.
- 통합 전에 모든 화면을 하나의 씬에 조립하지 않는다.
- 프로토타입에서 전체 특성 트리, 보스, 스킬, 자동 공격, 저장까지 한 번에 구현하지 않는다.

11. 팀 작업 시작 체크리스트

```
[ ] Contracts의 Snapshot 필드와 단위 합의  
[ ] CombatSandbox 생성  
[ ] ProgressionSandbox 생성  
[ ] FixedCombatSnapshot 테스트 더블 작성  
[ ] FakeProfile 테스트 더블 작성  
[ ] Combat 단독으로 60초 런 완주  
[ ] Progression 단독으로 구매 전후 Snapshot 검증  
[ ] App 통합 브랜치에서 런 시작과 결과창 연결  
[ ] t=60 경계와 중복 보상 테스트  
[ ] validate_collaboration.py 실행
```

현재 저장소에는 모듈 폴더와 asmdef, 문서가 준비되어 있으며 Combat 서비스와 Sandbox 인스턴스는 아직 구현 전이다. 따라서 첫 합류 목표는 이 문서의 60초 일반 런으로 고정한다.

참조: docs/collaboration/architecture.md, docs/collaboration/contracts.md, docs/v0.0.4/plan_core.md